

Traffic Light Module Control System

Arduino & 8086 Assembly Implementation
Computer Organization Course Project

Team Leader

- Mohammed Essam Eldin 2024/04701

Team Members

- Fatma Maged Mohammed 2024/05656
- Mahmoud Mohamed Hefny Salem 2024/06700
- Modather Abdalla 2024/03691
- Shahd Waleed Samy Hassan 2024/05549
- Ismail Sayed 2024/05812

Project Details:

Course: Computer Organization

Hardware: Traffic Light Arduino Module

Introduction

This project implements a traffic light control system using a pre-built Traffic Light Arduino Module with both 8086 assembly language and Arduino C. The module includes integrated red, yellow, and green LEDs with a control interface for managing traffic at an intersection with North-South (NS) and East-West (EW) directions.

8086 Assembly Implementation

This section explains the implementation of the traffic light control system using 8086 assembly language.

Processor Architecture Overview

The 8086 assembly language implementation leverages the fundamental architecture of the 8086 microprocessors. Understanding its registers and memory interaction is crucial for direct hardware control and optimized performance.



AX (Accumulator Register)

Used for arithmetic operations and data transfer.



CX (Count Register)

Used as a counter in loops and string operations.



DX (Data Register)

Used for I/O port addressing, multiplication, and division.



AH (High byte of AX)

Used for BIOS interrupts, particularly for video and keyboard operations.

Memory and I/O Interaction

The 8086 assembly directly addresses memory and I/O ports. This low-level access allows for precise control over the traffic light system's timing and state changes by writing directly to specific memory locations or I/O ports mapped to the traffic lights.

EMU 8086 Emulator: Step-by-Step Debugging

What is EMU 8086?

The EMU 8086 emulator is a powerful tool designed for learning and developing programs in 8086 assembly language. It provides a virtual environment to execute and debug assembly code, allowing users to observe register values, memory contents, and instruction flow in real-time.



Compile

Translate assembly code into machine-readable instructions using the integrated assembler.



Debug

Analyze program behavior by setting breakpoints, stepping through code, and inspecting registers and memory.



Execute

Run the compiled and debugged program to simulate 8086 processor operations and view output.

Traffic Light Arduino Module Setup

Hardware Setup

The Traffic Light Arduino Module is a pre-built, integrated solution that combines an Arduino Mega with a traffic light assembly featuring three color-coded LEDs (red, yellow, green) and a control interface. The module comes with pre-soldered connections and a breadboard-friendly design, eliminating the need for individual component assembly.

- Traffic Light Module
Integrated unit with LEDs and control interface.
- Arduino Mega
The central microcontroller for control.
- Pre-wired LED connections
Built-in wiring for Red, Yellow, and Green LEDs.
- Control pins
4 wires for power and signal connections.
- Breadboard
Optional for additional circuits and prototyping.

Assembly vs Arduino Comparison

Comparing the 8086 Assembly implementation with the Arduino C approach highlights the trade-offs between low-level control and high-level abstraction, each offering distinct advantages depending on the project requirements.

	8086 Assembly: Low-level control Direct hardware manipulation, no abstraction. Arduino C: High-level abstraction Simplified interaction with hardware via libraries.
	8086 Assembly: High Requires deep hardware knowledge and intricate coding. Arduino C: Lower Simpler syntax, pre-built functions, easier development.
	8086 Assembly: Steep Specific instruction sets, memory management details. Arduino C: Easier Based on C/C++, large community support and resources.
	8086 Assembly: Very Low Tied to specific processor architectures. Arduino C: Higher Adaptable across various microcontrollers with minimal changes.
	8086 Assembly: Performance-critical Firmware, OS kernels, embedded systems. Arduino C: Rapid Prototyping IoT projects, hobby electronics, automation tasks.

Testing and Validation

Thorough testing and validation are crucial for ensuring the correct and reliable operation of both the 8086 Assembly and Arduino implementations of the traffic light control system. This involves a systematic approach to verify functionality, debug issues, and ensure compliance with design specifications.

Assembly Testing (EMU 8086)

For the EMU 808G assembly code, verification focuses on instruction-level accuracy and precise hardware interaction within the emulated environment.



Verify Register Values

Inspect AX, BX, CX, DX at key program points.



Inspect Memory Locations

Ensure data is stored and retrieved correctly.



Confirm Flag Register States

Validate FLAGS after arithmetic and logical operations.



Check I/O Port Interactions

Ensure correct signals are sent to traffic lights.

Arduino Testing

The Arduino implementation requires testing at both the component level and the overall functional level to ensure the physical circuit and code work harmoniously.



Verify Light Cycle

Confirm NS and EW lights cycle correctly.



Measure Phase Durations

Accurately check timing for Green, Yellow, and Red.



Check for Inconsistencies

Monitor flickering or inconsistent LED behavior.



Test Power Cycle Recovery

Ensure graceful system recovery after power loss.

Conclusion and Key Learning Outcomes

This project successfully demonstrated the implementation of a traffic light control system using two distinct platforms: low-level 8086 Assembly for direct hardware interaction and high-level Arduino C for rapid prototyping and ease of development. By comparing these approaches, we gained valuable insights into the trade-offs between precision, complexity, and application suitability in embedded systems programming.

The comprehensive testing and validation methodologies applied to both implementations ensured their correct and reliable operation, highlighting the importance of systematic verification in hardware-software integration.

- Mastered low-level hardware control with 8086 Assembly
- Developed high-level control logic using Arduino C
- Understood the trade-offs between abstraction levels in embedded programming
- Applied systematic testing and debugging techniques for hardware-software systems
- Gained practical experience in real-world embedded systems applications